Aprendizaje de Máquina
Sesión 18: Aprendizaje de máquina para textos

Angel Rodríguez

Bag of Words

Una representación sencilla es el modelo

Bag of Words

Cada dimensión del vector corresponde a una palabra del vocabulario.

Si

$$V = [\text{machine}, \text{learning}, \text{is}, \text{useful}, \text{data}],$$

entonces

"machine learning is useful"

puede representarse como

$$x = [1, 1, 1, 1, 0].$$

¿Todas las palabras deberían pesar igual?

Supongamos que una palabra aparece muchas veces en **todos** los documentos.

¿Es realmente útil para distinguirlos?

Por ejemplo:

the, is, of, and, . . .

Queremos asignar mayor importancia a términos que

aparecen en un documento	pero no en todos los documentos

Esto motiva la representación

tf-idf

Term Frequency (TF)

La primera parte mide qué tan frecuente es una palabra dentro de un documento.

$$\mathrm{tf}_{t,d}$$

Una opción sencilla sería usar directamente la frecuencia de la palabra t en el documento d

$$\mathrm{tf}_{t,d} = \mathrm{count}(t, d).$$

Pero se utiliza una transformación logarítmica:

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t, d), & \mathrm{count}(t, d) > 0, \\ 0, & \text{en otro caso.} \end{cases}$$

Inverse Document Frequency (IDF)

Sea $N =$ número total de documentos y
$df_t =$ número de documentos que contienen el término t.
Entonces

$$\boxed{\text{idf}_t = \log_{10}\left(\frac{N}{df_t}\right)}$$

- Si t aparece en muchos documentos:

$$df_t \text{ grande} \quad \Rightarrow \quad idf_t \text{ pequeño.}$$

- Si t aparece en pocos documentos:

$$df_t \text{ pequeño} \quad \Rightarrow \quad idf_t \text{ grande.}$$

TF-IDF

Combinamos ambas cantidades:

$$\boxed{\text{tf-idf}(t, d) = \text{tf}_{t,d}\, \text{idf}_t}$$

La representación de un documento se convierte entonces en

$$x_d = [\text{tf-idf}(t_1, d), \ldots, \text{tf-idf}(t_p, d)].$$

Una palabra recibe un peso grande cuando

es frecuente en el documento y poco frecuente en la colección

Comparando documentos

Una vez que cada documento es un vector,

$$d_1 \longrightarrow x_1, \qquad d_2 \longrightarrow x_2,$$

podemos medir qué tan similares son.

Una de las medidas más utilizadas es

$$\boxed{\cos(x_1, x_2) = \frac{x_1^\top x_2}{\|x_1\|\|x_2\|}}$$

Es decir, el producto punto **normalizado por la longitud de los vectores**.

Ejemplo: búsqueda con TF-IDF

Queremos encontrar qué documentos son más relevantes para la consulta

Query: sweet love

en una colección de cuatro documentos:

d_1 : sweet sweet nurse love

d_2 : sweet sorrow

d_3 : how sweet is love

d_4 : nurse

El vocabulario de la colección es

$$\mathcal{V} = \{\text{sweet}, \text{nurse}, \text{love}, \text{how}, \text{sorrow}, \text{is}\}.$$

Compararemos la consulta con cada documento utilizando

$$\text{score}(q, d) = \cos(q, d) = \frac{q \cdot d}{\|q\| \, \|d\|}$$

donde los vectores contienen los pesos **TF-IDF**.

Otra idea: representar significado con vectores

Hasta ahora las dimensiones del vector corresponden directamente a palabras del vocabulario.

Pero también podemos intentar que la posición de un vector represente algo sobre su **significado**.

La idea fundamental es:

> palabras que aparecen en contextos similares $\Rightarrow$ significados relacionados

Esta es la base de la llamada

semántica vectorial.

Vectores basados en contexto

En lugar de representar un **documento**, podemos representar una **palabra**.

Por ejemplo:

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

Cada número indica cuántas veces la palabra objetivo aparece cerca de una palabra de contexto.

Contextos similares ⇒ vectores similares

Usando los vectores anteriores:

$$\cos(\text{cherry}, \text{information}) \approx 0{,}018,$$

mientras que

$$\cos(\text{digital}, \text{information}) \approx 0{,}996.$$

Por tanto,

digital está mucho más cerca de information

porque ambas palabras aparecen en contextos parecidos.

¿Qué es un embedding?

Un embedding representa una palabra mediante un vector como

$$\text{learning} \longrightarrow \begin{bmatrix} 0{,}21 \\ -0{,}72 \\ 0{,}18 \\ \vdots \\ 0{,}34 \end{bmatrix} \in \mathbb{R}^d.$$

A diferencia de Bag of Words:

- las dimensiones ya no corresponden directamente a palabras;
- los valores son números reales;
- pueden ser positivos o negativos;
- el vector es denso.

Word2Vec: idea general

Word2Vec aprende automáticamente embeddings a partir de los **contextos** en los que aparecen las palabras.

La idea general es:

Corpus $\longrightarrow$ contextos observados $\longrightarrow$ vectores aprendidos

Palabras utilizadas en contextos similares terminan con vectores cercanos en el espacio.

El significado aparece como una **relación geométrica**.

¿Por qué utilizar embeddings densos?

Una representación dispersa trata dimensiones distintas como independientes.

Por ejemplo,

$$\textit{car} \qquad \text{y} \qquad \textit{automobile}$$

corresponden a dimensiones diferentes del vocabulario.
Aunque sean palabras semánticamente similares, esta relación **no está explícitamente representada**.

Con embeddings densos buscamos que

$$\boxed{w_{\text{car}} \approx w_{\text{automobile}}}$$

y, por tanto,

$$\cos(w_{\text{car}}, w_{\text{automobile}}) \approx 1.$$

Además, trabajar con $d \ll |V|$ reduce considerablemente la dimensionalidad de la representación.

Word2Vec: aprender mediante una tarea auxiliar

La idea central de Word2Vec:

No aprendemos directamente “el significado” de una palabra.

En su lugar, construimos una **tarea de predicción** utilizando el propio corpus:

> ¿Es probable que la palabra *c* aparezca cerca de la palabra *w*?

Por ejemplo:

apricot y *jam*

probablemente aparecen juntas con mayor frecuencia que

apricot y *bed*.

Los parámetros aprendidos para resolver esta tarea se convierten en los **embeddings**.

Aprendizaje autosupervisado

Consideremos una oración del corpus:

. . . lemon, a [tablespoon of **apricot** jam, a] pinch . . .

Usando una ventana de contexto de ±2:

tablespoon	of	apricot	jam	a
c_1	c_2	w	c_3	c_4

El propio texto nos proporciona ejemplos positivos:

(apricot, tablespoon), (apricot, of), (apricot, jam), (apricot, a).

No necesitamos etiquetar manualmente los datos:

el corpus proporciona la supervisión

A esta idea se le llama **aprendizaje autosupervisado**.

Skip-gram: ejemplos positivos y negativos

Para entrenar el clasificador necesitamos dos tipos de ejemplos.

Ejemplos positivos

Palabras que realmente aparecen cerca:

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

Ejemplos negativos

Palabras seleccionadas aleatoriamente:

w	c_{neg}
apricot	aardvark
apricot	seven
apricot	forever
apricot	coaxial

Skip-gram: ejemplos positivos y negativos

Para entrenar el clasificador necesitamos dos tipos de ejemplos.

Ejemplos positivos
Palabras que realmente aparecen cerca:

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

Ejemplos negativos
Palabras seleccionadas aleatoriamente:

w	c_{neg}
apricot	aardvark
apricot	seven
apricot	forever
apricot	coaxial

El objetivo es aprender a distinguir

contextos reales vs. contextos aleatorios

Skip-gram: idea general

La intuición de **skip-gram** puede resumirse en cuatro pasos:

1. Considerar una **palabra objetivo** y una palabra vecina de su contexto como un **ejemplo positivo**.

2. Seleccionar aleatoriamente otras palabras del vocabulario para generar **ejemplos negativos**.

3. Utilizar **regresión logística** para entrenar un clasificador que distinga entre ejemplos positivos y negativos.

4. Utilizar los **pesos aprendidos** por el modelo como los **embeddings** de las palabras.

Skip-gram: el problema de clasificación

Dada una palabra objetivo w y una posible palabra de contexto c, queremos estimar

$$\boxed{P(+ \mid w, c)}$$

es decir,

$$P(c \text{ es un contexto real de } w).$$

La idea es que si dos palabras aparecen juntas frecuentemente, sus embeddings deberían ser similares.

$$\text{sim}(w, c) \approx c^T w.$$

Por tanto:

$$\begin{aligned} c^T w \text{ grande} &\Rightarrow \text{palabras compatibles,} \\ c^T w \text{ pequeño} &\Rightarrow \text{palabras poco compatibles.} \end{aligned}$$

Del producto punto a una probabilidad

El producto punto

$$c^T w$$

puede tomar cualquier valor entre $-\infty$ y ∞, por lo que no puede interpretarse directamente como una probabilidad.
Aplicamos la función sigmoide:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Entonces,

$$\boxed{P(+ \mid w, c) = \sigma(c^T w) = \frac{1}{1 + e^{-c^T w}}}$$

y

$$\boxed{P(- \mid w, c) = 1 - P(+ \mid w, c) = \sigma(-c^T w).}$$

Por tanto, el modelo utiliza la **similitud entre embeddings** para decidir si dos palabras forman un contexto plausible.

Skip-gram: múltiples palabras de contexto

Hasta ahora consideramos la probabilidad para una sola palabra de contexto:

$$P(+ \mid w, c_i) = \sigma(c_i^T w).$$

Skip-gram: múltiples palabras de contexto

Hasta ahora consideramos la probabilidad para una sola palabra de contexto:

$$P(+ \mid w, c_i) = \sigma(c_i^T w).$$

Sin embargo, una palabra objetivo w tiene varias palabras en su ventana de contexto:

$$c_1, c_2, \ldots, c_L.$$

Skip-gram hace la suposición simplificadora de que las palabras de contexto son **independientes**. Por tanto,

$$\boxed{P(+ \mid w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i^T w)}$$

Tomando logaritmo, el producto se convierte en una suma:

$$\boxed{\log P(+ \mid w, c_{1:L}) = \sum_{i=1}^{L} \log \sigma(c_i^T w)}$$

Así, la probabilidad del contexto depende de la **similitud entre el embedding de la palabra objetivo** y el embedding de **cada palabra de contexto**.

¿Qué está aprendiendo Word2Vec?

Skip-gram mantiene realmente **dos vectores por palabra**:

$$\boxed{w = \text{embedding como palabra objetivo}}$$

y

$$\boxed{c = \text{embedding como palabra de contexto.}}$$

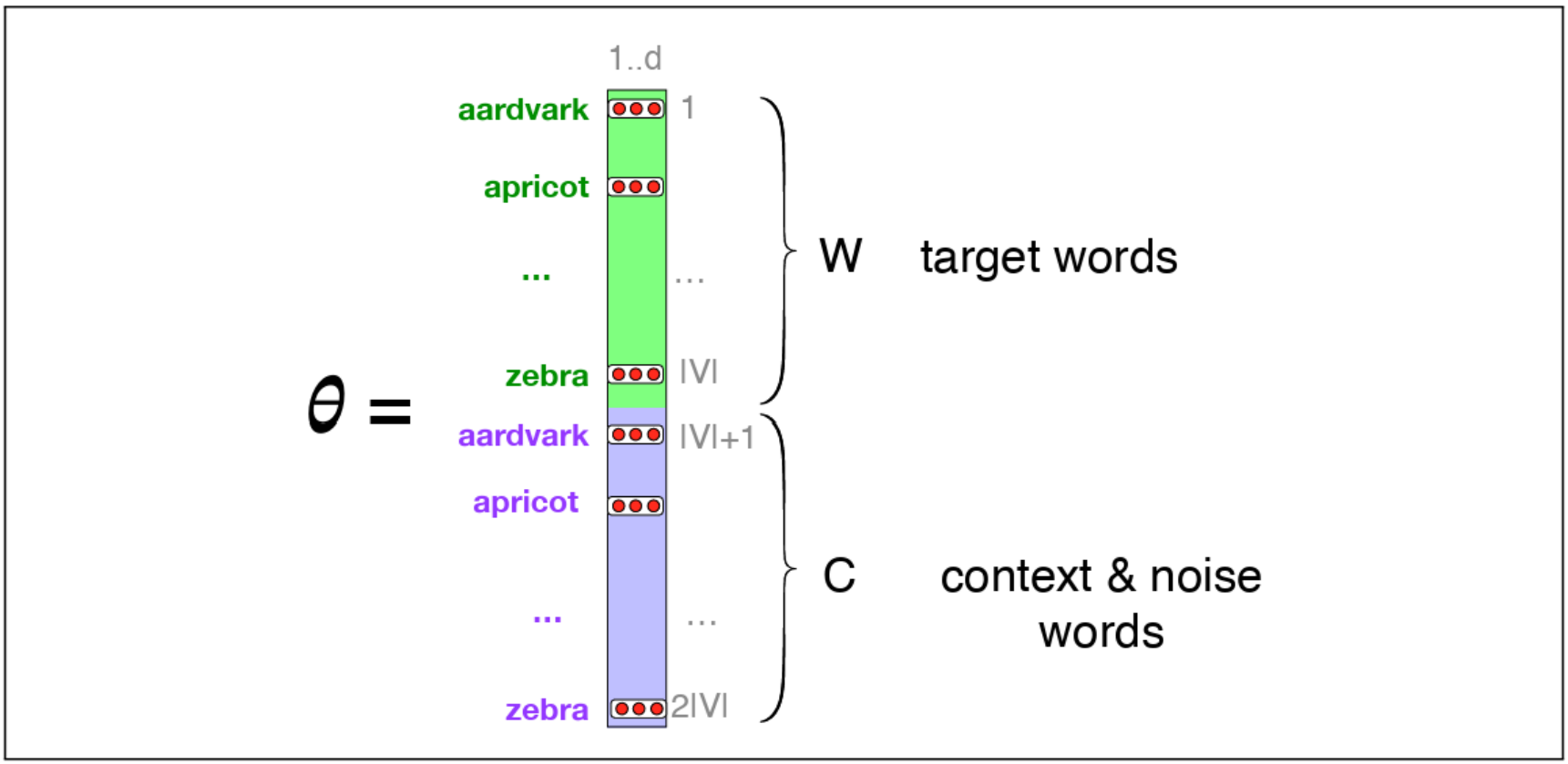
Por tanto, se aprenden dos matrices:

$$W \in \mathbb{R}^{|V| \times d}, \qquad C \in \mathbb{R}^{|V| \times d}.$$

W : embeddings de palabras objetivo
C : embeddings de palabras de contexto

Los valores de estas matrices son los **parámetros que se ajustan durante el entrenamiento**.

Skip-Gram


θ =
1..d
aardvark
1
apricot
...
...
zebra
|V|
W
target words
aardvark
|V|+1
apricot
...
...
zebra
2|V|
C
context & noise words

¿Cómo se aprenden los embeddings?

Inicialmente, los embeddings contienen valores aleatorios.

$$W, C \quad \longleftarrow \quad \text{valores aleatorios.}$$

Durante el entrenamiento queremos:

(w, c_{pos}) :	$w^T c_{\text{pos}}$	aumenta,
(w, c_{neg}) :	$w^T c_{\text{neg}}$	disminuye.

Geométricamente:

$$\text{apricot} \quad \underset{\text{acercar}}{\longleftrightarrow} \quad \text{jam}$$

mientras que

$$\text{apricot} \quad \underset{\text{separar}}{\longleftarrow \quad \longrightarrow} \quad \text{aardvark}$$

Negative sampling

Para cada par positivo

$$(w, c_{\text{pos}})$$

se generan k pares negativos:

$$(w, c_{\text{neg},1}), \ldots, (w, c_{\text{neg},k}).$$

Negative sampling

Para cada par positivo

$$(w, c_{\text{pos}})$$

se generan k pares negativos:

$$(w, c_{\text{neg},1}), \ldots, (w, c_{\text{neg},k}).$$

Las palabras negativas se muestrean del vocabulario.
Se utiliza una distribución modificada:

$$P_\alpha(w) = \frac{\text{count}(w)^\alpha}{\sum_{w'} \text{count}(w')^\alpha},$$

donde normalmente

$$\boxed{\alpha = 0{,}75}.$$

Negative sampling

Para cada par positivo

$$(w, c_{\text{pos}})$$

se generan k pares negativos:

$$(w, c_{\text{neg},1}), \ldots, (w, c_{\text{neg},k}).$$

Las palabras negativas se muestrean del vocabulario.
Se utiliza una distribución modificada:

$$P_\alpha(w) = \frac{\text{count}(w)^\alpha}{\sum_{w'} \text{count}(w')^\alpha},$$

donde normalmente

$$\boxed{\alpha = 0{,}75}.$$

Esto aumenta ligeramente la probabilidad relativa de seleccionar palabras poco frecuentes respecto a utilizar directamente sus frecuencias en el corpus.

Función de pérdida de Skip-gram

Consideremos una palabra central w, una palabra de contexto real c_{pos} y k palabras negativas $c_{\text{neg},1}, \ldots, c_{\text{neg},k}$.

Objetivo: aprender a distinguir los pares palabra-contexto reales de los pares negativos.

Función de pérdida de Skip-gram

Consideremos una palabra central w, una palabra de contexto real c_{pos} y k palabras negativas $c_{\text{neg},1}, \ldots, c_{\text{neg},k}$.

Objetivo: aprender a distinguir los pares palabra-contexto reales de los pares negativos.
Para ello, queremos que:

- El par positivo tenga una probabilidad alta de ser clasificado como vecino:

$$P(+ \mid w, c_{\text{pos}}) \longrightarrow 1.$$

- Cada par negativo tenga una probabilidad alta de ser clasificado como no vecino:

$$P(- \mid w, c_{\text{neg},i}) \longrightarrow 1.$$

Función de pérdida de Skip-gram

Consideremos una palabra central w, una palabra de contexto real c_{pos} y k palabras negativas $c_{\text{neg},1}, \ldots, c_{\text{neg},k}$.

Objetivo: aprender a distinguir los pares palabra-contexto reales de los pares negativos.
Para ello, queremos que:

- El par positivo tenga una probabilidad alta de ser clasificado como vecino:

$$P(+ \mid w, c_{\text{pos}}) \longrightarrow 1.$$

- Cada par negativo tenga una probabilidad alta de ser clasificado como no vecino:

$$P(- \mid w, c_{\text{neg},i}) \longrightarrow 1.$$

Suponiendo independencia condicional entre las clasificaciones, buscamos **maximizar**:

$$\boxed{P(+ \mid w, c_{\text{pos}}) \prod_{i=1}^{k} P(- \mid w, c_{\text{neg},i})}$$

Construcción de la función de pérdida

Queremos maximizar la probabilidad de clasificar correctamente los pares positivos y negativos:

$$P(+ \mid w, c_{\text{pos}}) \prod_{i=1}^{k} P(- \mid w, c_{\text{neg},i}).$$

Construcción de la función de pérdida

Queremos maximizar la probabilidad de clasificar correctamente los pares positivos y negativos:

$$P(+ \mid w, c_{\text{pos}}) \prod_{i=1}^{k} P(- \mid w, c_{\text{neg},i}).$$

Para formularlo como un problema de **minimización**, tomamos el logaritmo negativo:

$$\boxed{\mathcal{L} = -\log \left[P(+ \mid w, c_{\text{pos}}) \prod_{i=1}^{k} P(- \mid w, c_{\text{neg},i}) \right].}$$

Construcción de la función de pérdida

Queremos maximizar la probabilidad de clasificar correctamente los pares positivos y negativos:

$$P(+ \mid w, c_{\text{pos}}) \prod_{i=1}^{k} P(- \mid w, c_{\text{neg},i}).$$

Para formularlo como un problema de **minimización**, tomamos el logaritmo negativo:

$$\boxed{\mathcal{L} = -\log \left[P(+ \mid w, c_{\text{pos}}) \prod_{i=1}^{k} P(- \mid w, c_{\text{neg},i}) \right].}$$

Usando propiedades de logaritmo:

$$\mathcal{L} = -\left[\log P(+ \mid w, c_{\text{pos}}) + \sum_{i=1}^{k} \log P(- \mid w, c_{\text{neg},i}) \right].$$

Construcción de la función de pérdida

Queremos maximizar la probabilidad de clasificar correctamente los pares positivos y negativos:

$$P(+ \mid w, c_{\text{pos}}) \prod_{i=1}^{k} P(- \mid w, c_{\text{neg},i}).$$

Para formularlo como un problema de **minimización**, tomamos el logaritmo negativo:

$$\boxed{\mathcal{L} = -\log \left[P(+ \mid w, c_{\text{pos}}) \prod_{i=1}^{k} P(- \mid w, c_{\text{neg},i}) \right].}$$

Usando propiedades de logaritmo:

$$\mathcal{L} = -\left[\log P(+ \mid w, c_{\text{pos}}) + \sum_{i=1}^{k} \log P(- \mid w, c_{\text{neg},i}) \right].$$

Como $P(- \mid w, c) = 1 - P(+ \mid w, c)$, obtenemos:

$$\boxed{\mathcal{L} = -\left[\log P(+ \mid w, c_{\text{pos}}) + \sum_{i=1}^{k} \log\Big(1 - P(+ \mid w, c_{\text{neg},i})\Big) \right].}$$

Función de pérdida con Negative Sampling

En Skip-gram, la probabilidad de que dos palabras sean vecinas se modela mediante la función sigmoide:

$$P(+ \mid w, c) = \sigma(c^T w), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Función de pérdida con Negative Sampling

En Skip-gram, la probabilidad de que dos palabras sean vecinas se modela mediante la función sigmoide:

$$P(+ \mid w, c) = \sigma(c^T w), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Utilizando la identidad $1 - \sigma(z) = \sigma(-z)$, obtenemos:

$$\boxed{\mathcal{L} = -\left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w)\right].}$$

Función de pérdida con Negative Sampling

En Skip-gram, la probabilidad de que dos palabras sean vecinas se modela mediante la función sigmoide:

$$P(+ \mid w, c) = \sigma(c^T w), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Utilizando la identidad $1 - \sigma(z) = \sigma(-z)$, obtenemos:

$$\boxed{\mathcal{L} = -\left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w)\right].}$$

Interpretación de la minimización

- **Pares positivos:** aumentar el producto escalar $c_{\text{pos}}^T w$.
- **Pares negativos:** disminuir los productos escalares $c_{\text{neg},i}^T w$.

Así, el modelo aprende representaciones vectoriales que favorecen los contextos reales frente a los contextos negativos.

Entrenamiento de Word2Vec

La función de pérdida se minimiza utilizando **descenso por gradiente estocástico (SGD)**.

Para cada ejemplo de entrenamiento:

$$(w, c_{\text{pos}}, c_{\text{neg},1}, \ldots, c_{\text{neg},k})$$

se actualizan los embeddings correspondientes:

$$\boxed{\begin{array}{c} w \\ c_{\text{pos}} \\ c_{\text{neg},1}, \ldots, c_{\text{neg},k} \end{array}}$$

Entrenamiento de Word2Vec

La función de pérdida se minimiza utilizando **descenso por gradiente estocástico (SGD)**.

Para cada ejemplo de entrenamiento:

$$(w, c_{\text{pos}}, c_{\text{neg},1}, \ldots, c_{\text{neg},k})$$

se actualizan los embeddings correspondientes:

$$\boxed{\begin{array}{c} w \\ c_{\text{pos}} \\ c_{\text{neg},1}, \ldots, c_{\text{neg},k} \end{array}}$$

Después de procesar muchos contextos del corpus:

$$\boxed{\text{palabras con contextos similares} \longrightarrow \text{vectores similares}}$$

La tarea de clasificación es solo el mecanismo utilizado para **aprender los embeddings**.

Aprendizaje de los vectores en Skip-gram

Recordemos la función de pérdida de Skip-gram con Negative Sampling:

$$\mathcal{L} = -\left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w)\right].$$

Aprendizaje de los vectores en Skip-gram

Recordemos la función de pérdida de Skip-gram con Negative Sampling:

$$\mathcal{L} = -\left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w)\right].$$

Los vectores son parámetros del modelo que debemos ajustar para minimizar esta pérdida.

Objetivo del entrenamiento

- Aumentar el producto escalar entre la palabra central y su contexto positivo.
- Disminuir el producto escalar entre la palabra central y los contextos negativos.

Aprendizaje de los vectores en Skip-gram

Recordemos la función de pérdida de Skip-gram con Negative Sampling:

$$\mathcal{L} = -\left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w)\right].$$

Los vectores son parámetros del modelo que debemos ajustar para minimizar esta pérdida.

Objetivo del entrenamiento

- Aumentar el producto escalar entre la palabra central y su contexto positivo.
- Disminuir el producto escalar entre la palabra central y los contextos negativos.

Para realizar estos ajustes utilizamos el **descenso del gradiente estocástico (SGD)**.

Descenso del gradiente

Sea $\mathcal{L}(\boldsymbol{\theta})$ una función de pérdida que depende de los parámetros $\boldsymbol{\theta}$.

El **gradiente** es el vector de derivadas parciales:

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial \mathcal{L}}{\partial \theta_1} \\ \frac{\partial \mathcal{L}}{\partial \theta_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial \theta_d} \end{pmatrix}.$$

Descenso del gradiente

Sea $\mathcal{L}(\boldsymbol{\theta})$ una función de pérdida que depende de los parámetros $\boldsymbol{\theta}$.

El **gradiente** es el vector de derivadas parciales:

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial \mathcal{L}}{\partial \theta_1} \\ \frac{\partial \mathcal{L}}{\partial \theta_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial \theta_d} \end{pmatrix}.$$

El gradiente indica la dirección de mayor crecimiento local de la función.
Por tanto, para disminuir la pérdida nos movemos en la dirección opuesta:

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)})}$$

Descenso del gradiente

Sea $\mathcal{L}(\boldsymbol{\theta})$ una función de pérdida que depende de los parámetros $\boldsymbol{\theta}$.

El **gradiente** es el vector de derivadas parciales:

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial \mathcal{L}}{\partial \theta_1} \\ \frac{\partial \mathcal{L}}{\partial \theta_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial \theta_d} \end{pmatrix}.$$

El gradiente indica la dirección de mayor crecimiento local de la función.
Por tanto, para disminuir la pérdida nos movemos en la dirección opuesta:

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)})}$$

- $\eta > 0$: tasa de aprendizaje (*learning rate*).
- t: número de iteración.
- $-\nabla \mathcal{L}$: dirección de descenso.

Descenso del gradiente estocástico (SGD)

Supongamos que tenemos N ejemplos de entrenamiento y que la pérdida promedio es:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{j=1}^{N} \mathcal{L}_j(\boldsymbol{\theta}).$$

Descenso del gradiente estocástico (SGD)

Supongamos que tenemos N ejemplos de entrenamiento y que la pérdida promedio es:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{j=1}^{N} \mathcal{L}_j(\boldsymbol{\theta}).$$

Descenso del gradiente convencional:
Calcula el gradiente utilizando todos los ejemplos:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla J(\boldsymbol{\theta}^{(t)}).$$

Descenso del gradiente estocástico (SGD)

Supongamos que tenemos N ejemplos de entrenamiento y que la pérdida promedio es:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{j=1}^{N} \mathcal{L}_j(\boldsymbol{\theta}).$$

Descenso del gradiente convencional:
Calcula el gradiente utilizando todos los ejemplos:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla J(\boldsymbol{\theta}^{(t)}).$$

Descenso del gradiente estocástico (SGD):
Selecciona un ejemplo j y actualiza los parámetros utilizando solamente su pérdida:

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}_j(\boldsymbol{\theta}^{(t)})}$$

Descenso del gradiente estocástico (SGD)

Supongamos que tenemos N ejemplos de entrenamiento y que la pérdida promedio es:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{j=1}^{N} \mathcal{L}_j(\boldsymbol{\theta}).$$

Descenso del gradiente convencional:
Calcula el gradiente utilizando todos los ejemplos:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla J(\boldsymbol{\theta}^{(t)}).$$

Descenso del gradiente estocástico (SGD):
Selecciona un ejemplo j y actualiza los parámetros utilizando solamente su pérdida:

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}_j(\boldsymbol{\theta}^{(t)})}$$

¿Por qué estocástico?

Porque el ejemplo utilizado en cada actualización se selecciona aleatoriamente o mediante un orden aleatorio de los datos.
El gradiente calculado es una estimación del gradiente de la pérdida promedio.

Aplicación de SGD a Skip-gram

Para cada ejemplo de entrenamiento consideramos:

$$(w, c_{\text{pos}}, c_{\text{neg},1}, \ldots, c_{\text{neg},k}).$$

Aplicación de SGD a Skip-gram

Para cada ejemplo de entrenamiento consideramos:

$$(w, c_{\text{pos}}, c_{\text{neg},1}, \ldots, c_{\text{neg},k}).$$

La pérdida depende de los vectores:

$$\mathcal{L} = \mathcal{L}(w, c_{\text{pos}}, c_{\text{neg},1}, \ldots, c_{\text{neg},k}).$$

Para aplicar SGD necesitamos calcular:

$$\boxed{\frac{\partial \mathcal{L}}{\partial c_{\text{pos}}}, \qquad \frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}}, \qquad \frac{\partial \mathcal{L}}{\partial w}.}$$

Aplicación de SGD a Skip-gram

Para cada ejemplo de entrenamiento consideramos:

$$(w, c_{\text{pos}}, c_{\text{neg},1}, \ldots, c_{\text{neg},k}).$$

La pérdida depende de los vectores:

$$\mathcal{L} = \mathcal{L}(w, c_{\text{pos}}, c_{\text{neg},1}, \ldots, c_{\text{neg},k}).$$

Para aplicar SGD necesitamos calcular:

$$\boxed{\frac{\partial \mathcal{L}}{\partial c_{\text{pos}}}, \qquad \frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}}, \qquad \frac{\partial \mathcal{L}}{\partial w}.}$$

Regla de actualización

Para cada vector v involucrado:

$$v^{(t+1)} = v^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial v}.$$

Derivadas: resultados necesarios

Antes de derivar la pérdida, recordemos que:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Su derivada es:

Derivadas: resultados necesarios

Antes de derivar la pérdida, recordemos que:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Su derivada es:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z)).$$

Derivadas: resultados necesarios

Antes de derivar la pérdida, recordemos que:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Su derivada es:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z)).$$

Aplicando la regla de la cadena:

$$\frac{d}{dz} \log \sigma(z) = \frac{\sigma'(z)}{\sigma(z)} = 1 - \sigma(z).$$

Derivadas: resultados necesarios

Antes de derivar la pérdida, recordemos que:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Su derivada es:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z)).$$

Aplicando la regla de la cadena:

$$\frac{d}{dz} \log \sigma(z) = \frac{\sigma'(z)}{\sigma(z)} = 1 - \sigma(z).$$

De manera similar:

$$\frac{d}{dz} \log \sigma(-z) = -(1 - \sigma(-z)) = -\sigma(z).$$

Resumen:

$$\boxed{\begin{aligned} \frac{d}{dz} \log \sigma(z) &= 1 - \sigma(z), \\ \frac{d}{dz} \log \sigma(-z) &= -\sigma(z). \end{aligned}}$$

Derivada respecto al contexto positivo

Queremos calcular:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{pos}}} = \frac{\partial}{\partial c_{\text{pos}}} - \left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w) \right].$$

Solamente el primer término depende de c_{pos}. Definimos:

$$z = c_{\text{pos}}^T w.$$

Por la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{pos}}} = \frac{d[-\log \sigma(z)]}{dz} \frac{\partial z}{\partial c_{\text{pos}}}.$$

Derivada respecto al contexto positivo

Queremos calcular:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{pos}}} = \frac{\partial}{\partial c_{\text{pos}}} - \left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w) \right].$$

Solamente el primer término depende de c_{pos}. Definimos:

$$z = c_{\text{pos}}^T w.$$

Por la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{pos}}} = \frac{d[-\log \sigma(z)]}{dz} \frac{\partial z}{\partial c_{\text{pos}}}.$$

Como:

$$\frac{d[-\log \sigma(z)]}{dz} = \sigma(z) - 1, \qquad \frac{\partial z}{\partial c_{\text{pos}}} = w,$$

obtenemos:

$$\boxed{\frac{\partial \mathcal{L}}{\partial c_{\text{pos}}} = [\sigma(c_{\text{pos}}^T w) - 1] w.}$$

Derivada respecto a un contexto negativo

Para el contexto negativo i, queremos calcular:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}} = \frac{\partial}{\partial c_{\text{neg},i}} - \left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w) \right].$$

Derivada respecto a un contexto negativo

Para el contexto negativo i, queremos calcular:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}} = \frac{\partial}{\partial c_{\text{neg},i}} - \left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w) \right].$$

Definimos $z = c_{\text{neg},i}^T w$. Por la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}} = \frac{d[-\log \sigma(-z)]}{dz} \frac{\partial z}{\partial c_{\text{neg},i}}.$$

Derivada respecto a un contexto negativo

Para el contexto negativo i, queremos calcular:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}} = \frac{\partial}{\partial c_{\text{neg},i}} - \left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w) \right].$$

Definimos $z = c_{\text{neg},i}^T w$. Por la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}} = \frac{d[-\log \sigma(-z)]}{dz} \frac{\partial z}{\partial c_{\text{neg},i}}.$$

Utilizando las identidades anteriores:

$$\frac{d[-\log \sigma(-z)]}{dz} = \sigma(z),$$

y:

$$\frac{\partial z}{\partial c_{\text{neg},i}} = w.$$

Por tanto:

$$\boxed{\frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}} = \sigma(c_{\text{neg},i}^T w) w.}$$

Derivada respecto a la palabra central

Ahora derivamos la función completa respecto a w:

$$\mathcal{L} = -\log \sigma(c_{\text{pos}}^T w) - \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w).$$

Derivada respecto a la palabra central

Ahora derivamos la función completa respecto a w:

$$\mathcal{L} = -\log \sigma(c_{\text{pos}}^T w) - \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w).$$

Entonces, la derivada es:

$$\frac{\partial \mathcal{L}}{\partial w} = -\frac{\partial}{\partial w} \log \sigma(c_{\text{pos}}^T w) - \sum_{i=1}^{k} \frac{\partial}{\partial w} \log \sigma(-c_{\text{neg},i}^T w).$$

Derivada respecto a la palabra central

Ahora derivamos la función completa respecto a w:

$$\mathcal{L} = -\log \sigma(c_{\text{pos}}^T w) - \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w).$$

Entonces, la derivada es:

$$\frac{\partial \mathcal{L}}{\partial w} = -\frac{\partial}{\partial w} \log \sigma(c_{\text{pos}}^T w) - \sum_{i=1}^{k} \frac{\partial}{\partial w} \log \sigma(-c_{\text{neg},i}^T w).$$

Como:

$$\frac{\partial}{\partial w}(c^T w) = c,$$

obtenemos:

$$\boxed{\frac{\partial \mathcal{L}}{\partial w} = [\sigma(c_{\text{pos}}^T w) - 1]c_{\text{pos}} + \sum_{i=1}^{k} \sigma(c_{\text{neg},i}^T w) c_{\text{neg},i}.}$$

Actualización del contexto positivo

La regla general del descenso del gradiente es:

$$v^{(t+1)} = v^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial v}.$$

Actualización del contexto positivo

La regla general del descenso del gradiente es:

$$v^{(t+1)} = v^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial v}.$$

Sustituyendo el gradiente del contexto positivo:

$$\boxed{\begin{aligned} c_{\text{pos}}^{(t+1)} &= c_{\text{pos}}^{(t)} \\ &\quad - \eta[\sigma((c_{\text{pos}}^{(t)})^T w^{(t)}) - 1] w^{(t)}. \end{aligned}}$$

Actualización del contexto positivo

La regla general del descenso del gradiente es:

$$v^{(t+1)} = v^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial v}.$$

Sustituyendo el gradiente del contexto positivo:

$$\boxed{c_{\text{pos}}^{(t+1)} = c_{\text{pos}}^{(t)} - \eta[\sigma((c_{\text{pos}}^{(t)})^T w^{(t)}) - 1]w^{(t)}.}$$

Equivalentemente:

$$c_{\text{pos}}^{(t+1)} = c_{\text{pos}}^{(t)} + \eta(1 - \sigma(z_{\text{pos}}))w^{(t)},$$

donde:

$$z_{\text{pos}} = (c_{\text{pos}}^{(t)})^T w^{(t)}.$$

Interpretación

Como $0 < \sigma(z) < 1$, el coeficiente $1 - \sigma(z_{\text{pos}})$ es positivo. El vector positivo se modifica en la dirección del vector central, favoreciendo un producto escalar mayor.

Actualización de los contextos negativos

Para cada contexto negativo tenemos:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}} = \sigma(c_{\text{neg},i}^T w) w.$$

Actualización de los contextos negativos

Para cada contexto negativo tenemos:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}} = \sigma(c_{\text{neg},i}^T w) w.$$

Aplicando descenso del gradiente:

$$\boxed{c_{\text{neg},i}^{(t+1)} = c_{\text{neg},i}^{(t)} - \eta \sigma[(c_{\text{neg},i}^{(t)})^T w^{(t)}] w^{(t)}.}$$

Actualización de los contextos negativos

Para cada contexto negativo tenemos:

$$\frac{\partial \mathcal{L}}{\partial c_{\text{neg},i}} = \sigma(c_{\text{neg},i}^T w) w.$$

Aplicando descenso del gradiente:

$$\boxed{c_{\text{neg},i}^{(t+1)} = c_{\text{neg},i}^{(t)} - \eta \sigma[(c_{\text{neg},i}^{(t)})^T w^{(t)}] w^{(t)}.}$$

Interpretación

El coeficiente $\sigma(c_{\text{neg},i}^T w)$ es positivo. Entonces, restamos un múltiplo positivo de w al vector negativo.
Esto favorece que disminuya el producto escalar entre ambos vectores.

Cuanto mayor sea la probabilidad de clasificar incorrectamente el par negativo como positivo, mayor será la magnitud de esta corrección, para un vector central fijo (recordemos que $\sigma(c^T w) = P(+|w, c)$).

Actualización del vector central

El gradiente respecto al vector central es:

$$\frac{\partial \mathcal{L}}{\partial w} = [\sigma(c_{\text{pos}}^T w) - 1]c_{\text{pos}} + \sum_{i=1}^{k} \sigma(c_{\text{neg},i}^T w)c_{\text{neg},i}.$$

Actualización del vector central

El gradiente respecto al vector central es:

$$\frac{\partial \mathcal{L}}{\partial w} = [\sigma(c_{\text{pos}}^T w) - 1]c_{\text{pos}} + \sum_{i=1}^{k} \sigma(c_{\text{neg},i}^T w)c_{\text{neg},i}.$$

Aplicando la regla de actualización:

$$w^{(t+1)} = w^{(t)} - \eta\left[\left(\sigma((c_{\text{pos}}^{(t)})^T w^{(t)}) - 1\right)c_{\text{pos}}^{(t)} + \sum_{i=1}^{k} \sigma((c_{\text{neg},i}^{(t)})^T w^{(t)})c_{\text{neg},i}^{(t)}\right].$$

Actualización del vector central

El gradiente respecto al vector central es:

$$\frac{\partial \mathcal{L}}{\partial w} = [\sigma(c_{\text{pos}}^T w) - 1]c_{\text{pos}} + \sum_{i=1}^{k} \sigma(c_{\text{neg},i}^T w)c_{\text{neg},i}.$$

Aplicando la regla de actualización:

$$w^{(t+1)} = w^{(t)} - \eta\left[\left(\sigma((c_{\text{pos}}^{(t)})^T w^{(t)}) - 1\right)c_{\text{pos}}^{(t)} + \sum_{i=1}^{k} \sigma((c_{\text{neg},i}^{(t)})^T w^{(t)})c_{\text{neg},i}^{(t)}\right].$$

Interpretación

La actualización del vector central combina:

- Una contribución del contexto positivo, que favorece aumentar su producto escalar.
- Una contribución de cada contexto negativo, que favorece disminuir su producto escalar.

Entrenamiento de los embeddings

El modelo Skip-gram mantiene dos matrices:

$$W = \begin{pmatrix} - - - & w_1^T & - - - \\ & \vdots & \\ - - - & w_{|V|}^T & - - - \end{pmatrix}, \qquad C = \begin{pmatrix} - - - & c_1^T & - - - \\ & \vdots & \\ - - - & c_{|V|}^T & - - - \end{pmatrix}.$$

Entrenamiento de los embeddings

El modelo Skip-gram mantiene dos matrices:

$$W = \begin{pmatrix} - - - & w_1^T & - - - \\ & \vdots & \\ - - - & w_{|V|}^T & - - - \end{pmatrix}, \qquad C = \begin{pmatrix} - - - & c_1^T & - - - \\ & \vdots & \\ - - - & c_{|V|}^T & - - - \end{pmatrix}.$$

1. Inicializar aleatoriamente las matrices W y C.
2. Seleccionar un par positivo (w, c_{pos}) y sus k negativos.
3. Calcular la pérdida y sus gradientes.
4. Actualizar los vectores involucrados mediante las ecuaciones (5.25)–(5.27).
5. Repetir el procedimiento recorriendo las palabras del corpus.

Entrenamiento de los embeddings

El modelo Skip-gram mantiene dos matrices:

$$W = \begin{pmatrix} - - - & w_1^T & - - - \\ & \vdots & \\ - - - & w_{|V|}^T & - - - \end{pmatrix}, \qquad C = \begin{pmatrix} - - - & c_1^T & - - - \\ & \vdots & \\ - - - & c_{|V|}^T & - - - \end{pmatrix}.$$

1. Inicializar aleatoriamente las matrices W y C.
2. Seleccionar un par positivo (w, c_{pos}) y sus k negativos.
3. Calcular la pérdida y sus gradientes.
4. Actualizar los vectores involucrados mediante las ecuaciones (5.25)–(5.27).
5. Repetir el procedimiento recorriendo las palabras del corpus.

Representación final de las palabras

Una vez entrenado el modelo, podemos representar la palabra i mediante:

$$\boxed{e_i = w_i + c_i}$$

o utilizar solamente el vector central w_i.

Word2Vec aprende embeddings estáticos

Una característica importante de Word2Vec es que aprende **un único vector por palabra**.

$$\boxed{w \longrightarrow w}$$

Por ejemplo,

$$\text{bank} \longrightarrow w_{\text{bank}}.$$

Word2Vec aprende embeddings estáticos

Una característica importante de Word2Vec es que aprende **un único vector por palabra**.

$$\boxed{w \longrightarrow w}$$

Por ejemplo,

$$\text{bank} \longrightarrow w_{\text{bank}}.$$

El vector es el mismo independientemente del contexto:

"deposit money in the bank"

"sit on the river bank".

Word2Vec aprende embeddings estáticos

Una característica importante de Word2Vec es que aprende **un único vector por palabra**.

$$\boxed{w \longrightarrow w}$$

Por ejemplo,

$$\text{bank} \longrightarrow w_{\text{bank}}.$$

El vector es el mismo independientemente del contexto:

"deposit money in the bank"

"sit on the river bank".

Limitación

Word2Vec no genera una representación diferente para cada significado o contexto de una palabra.

Modelos posteriores utilizan **embeddings contextuales**, donde la representación depende de la oración en la que aparece la palabra.

Otros embeddings estáticos

Word2Vec no es el único método para obtener embeddings estáticos.

FastText
Representa una palabra utilizando también sus n-gramas de caracteres.
Por ejemplo, con $n = 3$:

$$\text{where} \longrightarrow \{<\texttt{wh}, \texttt{whe}, \texttt{her}, \texttt{ere}, \texttt{re}>\}.$$

Esto permite representar mejor palabras raras o incluso **palabras no observadas durante el entrenamiento**.

Otros embeddings estáticos

Word2Vec no es el único método para obtener embeddings estáticos.

FastText
Representa una palabra utilizando también sus *n*-gramas de caracteres.
Por ejemplo, con $n = 3$:

$$\text{where} \longrightarrow \{<\texttt{wh}, \texttt{whe}, \texttt{her}, \texttt{ere}, \texttt{re}>\}.$$

Esto permite representar mejor palabras raras o incluso **palabras no observadas durante el entrenamiento**.

GloVe
Aprende embeddings utilizando estadísticas globales de **coocurrencia de palabras** en el corpus.

Word2Vec, FastText y GloVe producen **embeddings estáticos**.

Word2Vec: resumen

Corpus → pares palabra–contexto → clasificación → embeddings

1. Elegimos una palabra objetivo w.
2. Las palabras cercanas generan ejemplos positivos.
3. Muestreamos palabras aleatorias como ejemplos negativos.
4. Comparamos los embeddings mediante el producto punto.
5. Ajustamos los vectores para distinguir ambos casos.

Word2Vec: resumen

Corpus → pares palabra–contexto → clasificación → embeddings

1. Elegimos una palabra objetivo w.
2. Las palabras cercanas generan ejemplos positivos.
3. Muestreamos palabras aleatorias como ejemplos negativos.
4. Comparamos los embeddings mediante el producto punto.
5. Ajustamos los vectores para distinguir ambos casos.

El resultado es una representación

$$w \longrightarrow \mathbf{w} \in \mathbb{R}^d$$

densa, de baja dimensión y capaz de capturar
relaciones semánticas entre palabras.

Resumen

Para aplicar aprendizaje de máquina a textos necesitamos

convertir texto en números

Dos grandes familias:

$$\text{Texto} \rightarrow \text{BoW} \rightarrow \text{TF-IDF}$$

representaciones dispersas

$$\text{Texto} \rightarrow \text{contextos} \rightarrow \text{embeddings}$$

representaciones densas

En ambos casos obtenemos vectores que podremos utilizar para

comparar, agrupar y clasificar textos.

Siguiente paso: agrupamiento de textos

Ahora podemos representar cada documento como un vector:

$$d_i \longrightarrow x_i.$$

Por tanto, los métodos de aprendizaje no supervisado que ya conocemos pueden aplicarse directamente:

$$\{x_1, \ldots, x_n\} \xrightarrow{K\text{-means}} \text{grupos de documentos}$$

4.3 Agrupamiento de textos

En el tema anterior transformamos cada documento en un vector:

$$d_i \longrightarrow x_i.$$

Por ejemplo, utilizando TF-IDF:

$$\boxed{\text{documentos} \longrightarrow \text{TF-IDF} \longrightarrow x_1, \ldots, x_n}$$

Ahora podemos aplicar métodos de **aprendizaje no supervisado**:

$$\boxed{\{x_1, \ldots, x_n\} \longrightarrow \text{clusters de documentos.}}$$

El algoritmo no cambia

En aprendizaje no supervisado ya estudiamos problemas de la forma

$$X = \begin{bmatrix} - & - & - & x_1 & - & - & - \\ - & - & - & x_2 & - & - & - \\ & & & \vdots & & & \\ - & - & - & x_n & - & - & - \end{bmatrix}.$$

Para datos tabulares, cada fila representaba una instancia.

Ahora:

cada fila representa un documento

y las características pueden corresponder a:

- frecuencias de palabras;
- valores TF-IDF;
- embeddings.

¿Para qué agrupar textos?

Supongamos que tenemos miles de documentos **sin etiquetas**.

Nos gustaría descubrir automáticamente documentos con características similares.

Por ejemplo:



El algoritmo no conoce previamente el significado de cada grupo.

Similitud entre textos

Dados dos documentos representados por

$$x_i, x_j,$$

podemos comparar sus direcciones mediante

$$\boxed{\cos(x_i, x_j) = \frac{x_i^\top x_j}{\|x_i\| \, \|x_j\|}}$$

Para vectores TF-IDF, los valores son no negativos, por lo que

$$0 \leq \cos(x_i, x_j) \leq 1.$$

$$\text{coseno grande} \quad \Rightarrow \quad \text{documentos similares.}$$

Interpretación geométrica

Dos documentos con vocabulario parecido producen vectores con direcciones similares.

$$\cos(\theta) = \frac{x^\top y}{\|x\|\|y\|}$$

$$\theta \approx 0^\circ \quad \Rightarrow \quad \cos(\theta) \approx 1$$

documentos muy similares

$$\theta \approx 90^\circ \quad \Rightarrow \quad \cos(\theta) \approx 0$$

documentos poco similares

La comparación depende principalmente de la **dirección**, no de la longitud del vector.

De similitud a distancia

Si queremos expresar la relación como una distancia, una opción habitual es

$$\boxed{d_{\cos}(x, y) = 1 - \cos(x, y)}$$

Así:

$$\cos(x, y) \approx 1 \Rightarrow d_{\cos}(x, y) \approx 0,$$

$$\cos(x, y) \approx 0 \Rightarrow d_{\cos}(x, y) \approx 1.$$

Textos más parecidos $\Rightarrow$ menor distancia.

¿Qué hacía K-means?

K-means busca minimizar la suma de las distancias cuadráticas dentro de cada grupo:

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \boldsymbol{\mu}_k\|^2$$

Por tanto, intenta construir clusters donde

los documentos queden cerca de su centroide.

La interpretación del cluster dependerá de las características utilizadas para representar los textos.

Distancia euclidiana y coseno

K-means utiliza normalmente distancia euclidiana.
Pero en texto usamos frecuentemente similitud coseno.

Si normalizamos

$$\|x\| = \|y\| = 1,$$

entonces

$$\begin{aligned}\|x - y\|^2 &= \|x\|^2 + \|y\|^2 - 2x^\top y \\ &= 2 - 2\cos(x, y) \\ &= 2\Big(1 - \cos(x, y)\Big).\end{aligned}$$

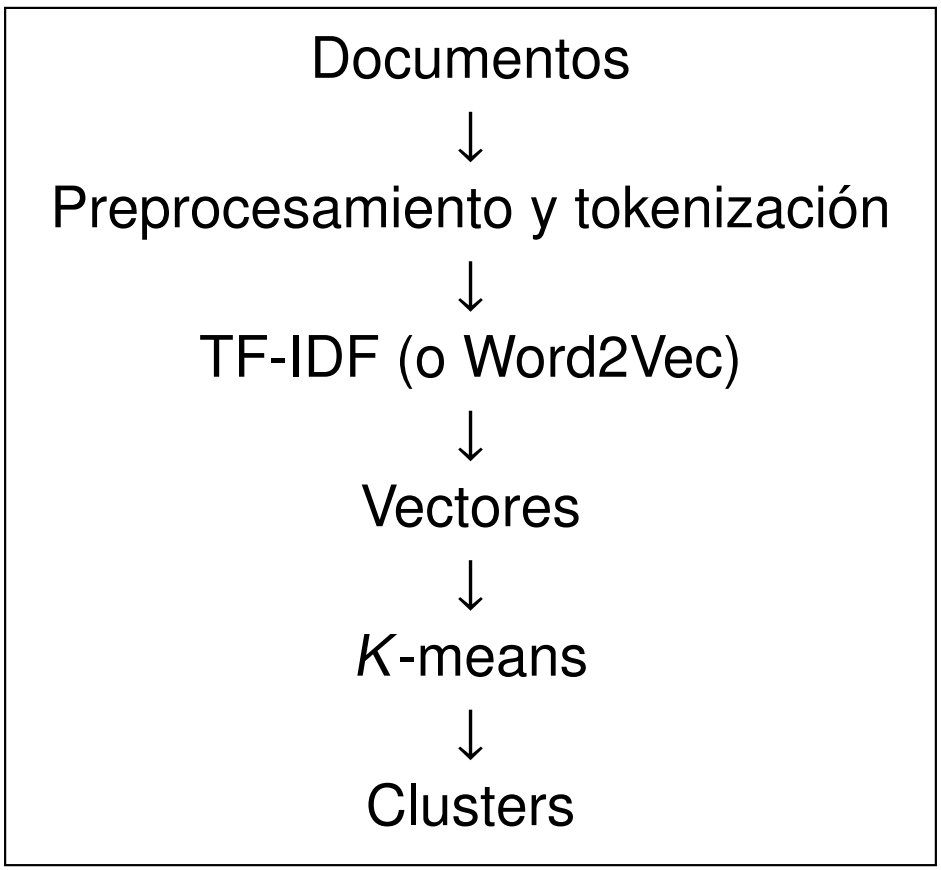
Por tanto, para vectores normalizados:

menor distancia euclidiana $\iff$ mayor similitud coseno.

K-means esférico.

Pipeline para agrupar textos

El procedimiento completo puede escribirse como



Ejemplo conceptual

Supongamos que tenemos noticias sobre

fútbol	acciones	inteligencia artificial
tenis	bancos	redes neuronales
baloncesto	mercados	algoritmos

Después de vectorizar y agrupar podríamos obtener

$$C_1, \quad C_2, \quad C_3.$$

Posteriormente, al inspeccionar los documentos, podríamos interpretar los grupos como

deportes, finanzas, tecnología.

K-means no produce esos nombres.

¿Cómo interpretamos un cluster?

Si utilizamos TF-IDF, cada dimensión corresponde a una palabra.

El centroide

$$\boldsymbol{\mu}_k = [\mu_{k1}, \ldots, \mu_{kp}]$$

también tiene una componente para cada término.

Podemos buscar las componentes más grandes:

$$\boxed{\mu_{kj} \text{ grande} \Rightarrow \text{término característico del cluster}}$$

Por ejemplo:

$$C_1 : \quad \{\text{goal}, \text{match}, \text{team}, \text{player}, \ldots\}$$

puede sugerir un grupo relacionado con deportes.

Implementación con Scikit-Learn

Una implementación básica puede utilizar:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(documents)

model = KMeans(
    n_clusters=3,
    random_state=42
)

labels = model.fit_predict(X)
```

El resultado

`labels`

indica el cluster asignado a cada documento.

Interpretando los grupos

Podemos inspeccionar las palabras asociadas con las componentes más grandes de cada centroide:

```
terms = vectorizer.get_feature_names_out()

for k, centroid in enumerate(model.cluster_centers_):
    indices = centroid.argsort()[-10:][::-1]
    words = terms[indices]

    print(k, words)
```

Esto puede producir resultados como:

```
0: team game player match goal ...
1: market bank stock price economy ...
2: model data algorithm neural learning ...
```

¿Cuántos clusters utilizar?

K-means requiere especificar previamente

$$K.$$

Las técnicas estudiadas anteriormente continúan siendo aplicables:

- curva de inercia;
- método del codo;
- coeficiente de silueta;
- interpretación de los grupos obtenidos.

La elección de K sigue siendo parte del problema.

Coeficiente de silueta

Siguiente clase

La representación cambia los clusters

Consideremos los mismos documentos.

Podemos aplicar K-means sobre

$$X_{\text{BoW}}, \qquad X_{\text{TF-IDF}}, \qquad X_{\text{embedding}}.$$

Aunque el algoritmo sea el mismo,

las geometrías del espacio son diferentes

y por tanto pueden obtenerse agrupamientos distintos.

La representación forma parte fundamental del modelo.

TF-IDF o embeddings

	TF-IDF	**Embeddings**
Tipo	Disperso	Denso
Dimensiones	Palabras	Latentes
Similitud	Léxica	Puede capturar semántica
Interpretación	Más directa	Menos directa

Con TF-IDF, dos documentos son similares principalmente si comparten términos.

Con embeddings, documentos con vocabulario distinto pueden quedar próximos si sus representaciones capturan información semántica.

¿Qué significan los clusters?

Un cluster no es necesariamente una categoría “verdadera” del conjunto de datos.

El resultado depende de

preprocesamiento
representación
medida geométrica
K
algoritmo

Por eso es necesario

inspeccionar e interpretar los resultados.

Actividad: agrupamiento de documentos

Objetivo

Agrupar documentos según su contenido utilizando **TF-IDF y K-means**, sin conocer previamente las categorías de los textos.

1. Representar los documentos mediante TF-IDF.
2. Aplicar K-means para distintos valores de K.
3. Evaluar los agrupamientos mediante inercia y coeficiente de silueta.
4. Identificar las palabras más importantes de cada centroide.
5. Interpretar los temas encontrados.

Pregunta: ¿Podemos descubrir automáticamente los temas de una colección de documentos?

Dataset: 20 Newsgroups

Utilizaremos una colección de mensajes de discusión sobre diferentes temas.

Para simplificar, seleccionamos cuatro categorías:

- Ciencia espacial.
- Béisbol.
- Gráficos por computadora.
- Política de Medio Oriente.

Importante

Las categorías originales no se utilizan para entrenar K-means.
El objetivo es descubrir grupos utilizando únicamente el contenido de los documentos.

¿Cuántos grupos esperamos encontrar?

Paso 1: cargar los documentos

```
from sklearn.datasets import fetch_20newsgroups

categorias = ["sci.space","rec.sport.baseball", "comp.graphics","talk.
    politics.mideast"]

datos = fetch_20newsgroups(
    subset="train",
    categories=categorias,
    remove=("headers", "footers", "quotes"),
    shuffle=True,
    random_state=42
)

documentos = datos.data

print("Documentos:", len(documentos))
print(documentos[11][:500])
```

Paso 2: representación mediante TF-IDF

Transformamos cada documento en un vector numérico:

$$d_i \longrightarrow x_i = (x_{i1}, x_{i2}, \ldots, x_{ip}).$$

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizador = TfidfVectorizer(stop_words="english",min_df=5,max_df=0.8,
     max_features=5000,norm="l2")

X = vectorizador.fit_transform(documentos)

print(X.shape)
```

Observaciones

- Cada fila representa un documento.
- Cada columna representa un término.
- Los vectores están normalizados: $\|x_i\|_2 = 1$ para documentos no vacíos.

Paso 3: aplicar K-means

Aplicamos K-means para diferentes números de grupos, $K = 2, 3, \ldots, 8$.

```
from sklearn.cluster import KMeans

modelos = {}

for k in range(2, 9):
    modelo = KMeans(n_clusters=k,random_state=42,n_init=10)

    modelo.fit(X)
    modelos[k] = modelo

    print(k, modelo.inertia_)
```

Recordatorio

K-means clásico minimiza la suma de distancias euclidianas al cuadrado:

$$J = \sum_{i=1}^{N} \|x_i - \boldsymbol{\mu}_{z_i}\|^2.$$

Paso 4: evaluar los agrupamientos

Compararemos dos criterios:

- **Inercia:** queremos valores pequeños.
- **Silueta:** preferimos valores cercanos a 1.

```
from sklearn.metrics import silhouette_score

inercias = []
siluetas = []

for k, modelo in modelos.items():
    inercias.append(modelo.inertia_)

    s = silhouette_score(X,modelo.labels_,metric="euclidean",sample_size=
         min(1000, X.shape[0]),random_state=42)

    siluetas.append(s)
```

Pregunta

¿El valor de K con mejor silueta coincide con el sugerido por el método del codo?

Paso 5: visualizar los resultados

```
import matplotlib.pyplot as plt

valores_k = list(modelos.keys())

plt.figure()
plt.plot(valores_k, inercias, marker="o")
plt.xlabel("Numero de clusters K")
plt.ylabel("Inercia")
plt.title("Metodo del codo")
plt.show()

plt.figure()
plt.plot(valores_k, siluetas, marker="o")
plt.xlabel("Numero de clusters K")
plt.ylabel("Silueta")
plt.title("Coeficiente de silueta")
plt.show()
```

Discusión

La inercia generalmente disminuye al aumentar K. Por eso no basta con elegir el valor que produce la menor inercia.

Paso 6: interpretar los clusters

Seleccionamos un valor de K y examinamos las palabras con mayor peso en cada centroide.

```
import numpy as np

modelo = modelos[4]

terminos = vectorizador.get_feature_names_out()

for j, centroide in enumerate(modelo.cluster_centers_):
    indices = np.argsort(centroide)[::-1][:10]
    palabras = terminos[indices]

    print(f"Cluster {j}:")
    print(", ".join(palabras))
```

Preguntas

1. ¿Qué tema parece representar cada cluster?
2. ¿Qué palabras permiten identificarlo?
3. ¿Existen clusters que mezclen varios temas?

Paso 7: comparar con las categorías reales

Ahora revelamos las categorías originales y comparamos los resultados.

```
import pandas as pd

etiquetas_reales = [
    datos.target_names[i]
    for i in datos.target
]

tabla = pd.crosstab( pd.Series(modelo.labels_, name="Cluster"),pd.Series(
     etiquetas_reales, name="Tema real") )

print(tabla)
```

Preguntas

- ¿Los clusters coinciden con los temas originales?
- ¿Qué categorías se confunden más?
- ¿Por qué los identificadores de los clusters no tienen que coincidir con las etiquetas?

Extensión: modificar la representación

Nuevo experimento

Repetir el agrupamiento sustituyendo TF-IDF por otra representación de los documentos.

1. Utilizar `CountVectorizer` para representar frecuencias de términos.
2. Normalizar los vectores mediante norma L_2.
3. Aplicar nuevamente K-means.
4. Comparar la silueta y la interpretación de los clusters.

Preguntas finales

¿La representación de los documentos influye en los grupos encontrados?

¿Podríamos utilizar embeddings en lugar de TF-IDF?

Resumen

El agrupamiento de textos requiere dos pasos fundamentales:

$$\boxed{\text{Texto} \longrightarrow \text{representación vectorial} \longrightarrow \text{clustering}}$$

Por ejemplo:

$$\boxed{\text{documentos} \rightarrow \text{TF-IDF} \rightarrow K\text{-means} \rightarrow \text{clusters}}$$

Las ideas principales son:

- cada documento se representa como un vector;
- la similitud coseno permite comparar vectores de texto;
- K-means agrupa vectores alrededor de centroides;
- los clusters deben interpretarse posteriormente.

Siguiente paso: clasificación de textos

Hasta ahora:

documentos sin etiquetas $\longrightarrow$ clusters

¿Qué ocurre si conocemos una etiqueta para cada documento?

reseña positiva reseña negativa	o	deportes finanzas tecnología

Entonces podemos utilizar **aprendizaje supervisado**.

¡Gracias!